Code Appendix

2025-12-03

```r
## ================================
## 0. Load needed packages and data
## ================================
# set up the environment
setwd("~/Dropbox/School/CU/fall 2025/BIOS 6618 adv biostatistical method/final project/surgery_timing_ou
# Helper function: install if not already available
load_or_install <- function(pkg){
  if (!requireNamespace(pkg, quietly = TRUE)) {
    install.packages(pkg)
  }
  library(pkg, character.only = TRUE)
}

# List of needed packages
packages <- c("dplyr", "ggplot2",  "kableExtra", "webshot2","magick","knitr","tidyverse",
              "tidyr","ggh4x","GGally","visdat","car","fastDummies","naniar",
              "rms","Hmisc","patchwork","forcats","xfun","gtsummary","gt")
# Load or install each
lapply(packages, load_or_install)
data <- read.csv("./data/Surgery_Timing.csv")

##############################################################
## 1. Define variable groups (by role & type)
##############################################################

# Primary exposure & outcomes
var_exposure   <- "hour"
var_outcome   <- "mort30"

# Comorbidity indicators (0/1)
vars_comorbidity <- c(
  "baseline_cancer",
  "baseline_cvd",
  "baseline_dementia",
  "baseline_diabetes",
  "baseline_digestive",
  "baseline_osteoart",
  "baseline_psych",
  "baseline_pulmonary"
)

# Continuous numeric variables for EDA
vars_continuous <- c(
  "age",
```

```r
  "bmi",
  "baseline_charlson",
  "mortality_rsi",
  "complication_rsi",
  "ccsMort30Rate",
  "ccsComplicationRate",
  "hour"
)

# Binary variables (0/1)
vars_binary <- c(
  vars_comorbidity,
  var_outcome
)

# Categorical variables
vars_categorical <- c(
  "gender",
  "race",
  "asa_status",
  "ahrq_ccs",
  "dow",
  "month",
  "moonphase"
)
n_cols <- ncol(data)
print(length(vars_categorical) + length(vars_binary) + length(vars_continuous) == n_cols)  # should equ
vars_categorical_binary <- c(vars_categorical, vars_binary)
##############################################################
## 2. Type conversion
##############################################################

data <- data %>%
  # Convert binary variables to factors
  mutate(across(all_of(vars_binary), ~ as.numeric(.))) %>%
  # Convert categorical variables to factors
  mutate(across(all_of(vars_categorical), ~ as.factor(.))) %>%
  # convert continuous variables to numeric
  mutate(across(all_of(vars_continuous), ~ as.numeric(.))
  )
```

```r
cat("=== Structure ===\n")
str(data)
summary(data)

##############################################################
# logic cleaning
##############################################################
# keep age >= 18
orin_dim <- dim(data)
# find mort30 for age < 18
mort_young <- data %>%
  dplyr::filter(age < 18) %>%
```

```r
  dplyr::select(mort30)
cat("30-day mortality for patients age < 18:\n")
print(table(mort_young$mort30))
print(paste0("Original data dimensions: ", orin_dim[1], " rows, ", orin_dim[2], " columns"))
data <- data %>%
  filter(age >= 18)
cat("Data dimensions after filtering age >= 18:\n")
print(dim(data))
removed_rows <- orin_dim[1] - nrow(data)
cat(paste0("Number of rows removed due to age < 18: ", removed_rows, "\n"))
# check age summary
summary(data$age)


##############################################################
## multicollinearity check for continuous variables
##############################################################
# calculate vif for continuous variables
vif_data <- data %>%
  dplyr::select(all_of(c(vars_continuous, "mort30"))) %>%
  na.omit()
vif_model <- lm(mort30 ~ ., data = vif_data)
vif_values <- vif(vif_model)
vif_results <- data.frame(
  variable = names(vif_values),
  vif      = as.numeric(vif_values),
  stringsAsFactors = FALSE
)
print(vif_results)
# return variables with vif > 5
high_vif_vars <- vif_results$variable[vif_results$vif > 10]
cat("Continuous variables with high VIF (> 5):\n")
print(high_vif_vars)

# calculate correlation matrix for continuous variables
cor_data <- data %>%
  dplyr::select(all_of(vars_continuous)) %>%
  na.omit()
cor_matrix <- cor(cor_data)
print(cor_matrix)
# return variable pairs with high correlation (> 0.7)
# remove duplicate pairs and self-correlation
high_cor_results <- data.frame(
  variable1 = character(),
  variable2 = character(),
  correlation = numeric(),
  stringsAsFactors = FALSE
)
for (i in 1:(ncol(cor_matrix) - 1)) {
  for (j in (i + 1):ncol(cor_matrix)) {
    cor_value <- cor_matrix[i, j]
    if (abs(cor_value) > 0.7) {
      high_cor_results <- rbind(
```

```
        high_cor_results,
        data.frame(
          variable1   = colnames(cor_matrix)[i],
          variable2   = colnames(cor_matrix)[j],
          correlation = cor_value,
          stringsAsFactors = FALSE
        )
      )
    }
  }
}
print(high_cor_results)
# Based on the results, consider removing one variable from each highly correlated pair
numeric_vars_remove <- c("bmi","complication_rsi","ccsComplicationRate")
# final continuous variables to consider
numeric_vars_consider <- setdiff(vars_continuous, numeric_vars_remove)
cat("Variable pairs with high correlation (> 0.7):\n")
print(high_cor_results)
cat("Removing the following continuous variables due to high multicollinearity:\n")
print(numeric_vars_remove)
cat("Continuous variables to consider after removing multicollinear variables:\n")
print(numeric_vars_consider)

##############################################################
#missing value imputation
##############################################################
# check missingness
missing_summary <- data.frame(
  variable       = character(),
  missing_count  = numeric(),
  missing_percent = numeric(),
  stringsAsFactors = FALSE
)
for (v in colnames(data)) {
  missing_count <- sum(is.na(data[[v]]))
  missing_percent <- missing_count / nrow(data) * 100
  missing_summary <- rbind(
    missing_summary,
    data.frame(
      variable        = v,
      missing_count   = missing_count,
      missing_percent = missing_percent,
      stringsAsFactors = FALSE
    )
  )
}
# check missingness by mort30 and pivot by mort30
missing_by_mort30 <- data %>%
  group_by(mort30) %>%
  summarise(across(everything(), ~ sum(is.na(.)))) %>%
  tidyr::pivot_longer(
    cols = -mort30,
    names_to = "variable",
```

```r
    values_to = "missing_count"
  ) %>%
  tidyr::pivot_wider(
    names_from = mort30,
    values_from = missing_count,
    names_prefix = "mort30_"
  )
missing_summary <- missing_summary %>%
  left_join(missing_by_mort30, by = "variable")
print(missing_summary)

##############################################
# finalize variable lists for modeling
##############################################
vars_to_check <- c("age", "gender", "asa_status")
vars_to_drop  <- c("bmi", "race",
                   "ccsComplicationRate","complication_rsi"
                   )
cat("Variables to drop due to  high correlation or high missingness:\n")
print(vars_to_drop)
data_clean <- data %>%
  drop_na(all_of(vars_to_check)) %>%
  dplyr::select(-all_of(vars_to_drop))
cat("Data dimensions after dropping missing values in key variables and unnecessary variables:\n")
print(dim(data_clean))
print(length(vars_to_drop))
length(vars_to_drop) + ncol(data_clean) == ncol(data)  # should equal ncol(data)
# final variable lists
final_vars_categorical <- setdiff(vars_categorical, vars_to_drop)
final_vars_binary      <- setdiff(vars_binary, vars_to_drop)
final_vars_continuous  <- numeric_vars_consider
cat("Final categorical variables:\n")
print(final_vars_categorical)
cat("Final binary variables:\n")
print(final_vars_binary)
cat("Final continuous variables:\n")
print(final_vars_continuous)
# check missingness again
final_missing_summary <- data.frame(
  variable       = character(),
  missing_count  = numeric(),
  missing_percent = numeric(),
  stringsAsFactors = FALSE
)
for (v in colnames(data_clean)) {
  missing_count <- sum(is.na(data_clean[[v]]))
  missing_percent <- missing_count / nrow(data_clean) * 100
  final_missing_summary <- rbind(
    final_missing_summary,
    data.frame(
      variable        = v,
      missing_count   = missing_count,
      missing_percent = missing_percent,
```

```
      stringsAsFactors = FALSE
    )
  )
}
print(final_missing_summary)

##############################################################
# combine ahrq_ccs levels for modeling
##############################################################
data_clean <- data_clean %>%
  mutate(
    procedure_group = case_when(
      ahrq_ccs %in% c("Colorectal resection",
                      "Gastrectomy; partial and total",
                      "Small bowel resection",
                      "Inguinal and femoral hernia repair",
                      "Other hernia repair") ~ "Gastrointestinal & Hernia",

      ahrq_ccs %in% c("Lumpectomy; quadrantectomy of breast",
                      "Mastectomy") ~ "Breast",

      ahrq_ccs %in% c("Arthroplasty knee",
                      "Hip replacement; total and partial",
                      "Laminectomy; excision intervertebral disc",
                      "Spinal fusion") ~ "Orthopedic & Spine",

      ahrq_ccs %in% c("Hysterectomy; abdominal and vaginal",
                      "Oophorectomy; unilateral and bilateral",
                      "Other excision of cervix and uterus",
                      "Repair of cystocele and rectocele; obliteration of vaginal vault") ~ "Gynecologic

      ahrq_ccs %in% c("Endoscopy and endoscopic biopsy of the urinary tract",
                      "Nephrectomy; partial or complete",
                      "Open prostatectomy",
                      "Transurethral resection of prostate (TURP)") ~ "Urologic",

      ahrq_ccs %in% c("Thyroidectomy; partial or complete") ~ "Endocrine",

      ahrq_ccs %in% c("Plastic procedures on nose") ~ "Plastic",

      TRUE ~ "Other non-major procedures"
    )
  )
# check procedure group by original ahrq_ccs
print(unique(data_clean$procedure_group))
procedure_check <- data_clean %>%
  group_by(procedure_group,ahrq_ccs) %>%
  summarise(count = n()) %>%
  arrange(procedure_group)
print(procedure_check, n=length(unique(data_clean$ahrq_ccs)))

#plot histogram of mort30 by procedure_group
ggplot(data_clean, aes(x=procedure_group, fill=factor(mort30))) +
```

```r
  geom_bar(position="dodge") +
  labs(title="30-day Mortality by Procedure Group", x="Procedure Group", y="Count", fill="30-day Mortali
  theme_minimal()
```

```r
# summarize mort30 by procedure_group
# check sample size (n<100?) and death(n<10?) in each procedure_group
group_summary <- data_clean %>%
  group_by(procedure_group) %>%
  summarise(
    n          = n(),
    deaths     = sum(mort30, na.rm = TRUE),
    mort_rate  = mean(mort30, na.rm = TRUE),
    .groups    = "drop"
  ) %>%
  arrange(desc(n))

group_summary

# check heterogeneity within each group
# within the group, are mortality rates similar across different ahrq_ccs?
within_group <- data_clean %>%
  group_by(procedure_group, ahrq_ccs) %>%
  summarise(
    n          = n(),
    deaths     = sum(mort30, na.rm = TRUE),
    mort_rate  = mean(mort30, na.rm = TRUE),
    .groups    = "drop"
  ) %>%
  arrange(procedure_group, desc(n))

print(within_group, n=27)

ggplot(group_summary, aes(x = procedure_group, y = mort_rate)) +
  geom_col() +
  geom_text(
    aes(label = sprintf("%.2f%%", mort_rate * 100)),
    vjust = -0.3,
    size  = 3
  ) +
  labs(
    title = "30-day mortality rate by procedure group",
    x     = "Procedure group",
    y     = "Mortality rate"
  ) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

```r
# combine group with < 6 deaths
data_clean <- data_clean %>%
  mutate(
    procedure = ifelse(
      procedure_group %in% group_summary$procedure_group[group_summary$deaths < 6],
      "Other non-major procedures",
```

```r
      procedure_group
    )
  )
# check new procedure group summary
new_group_summary <- data_clean %>%
  group_by(procedure) %>%
  summarise(
    n          = n(),
    deaths     = sum(mort30, na.rm = TRUE),
    mort_rate  = mean(mort30, na.rm = TRUE),
    .groups    = "drop"
  ) %>%
  arrange(desc(n))
print(new_group_summary)

#check finalized group, procedure group and ahrq_ccs
final_procedure_check <- data_clean %>%
  group_by(procedure, procedure_group,ahrq_ccs) %>%
  summarise(count = n()) %>%
  arrange(procedure)
print(final_procedure_check, n=length(unique(data_clean$ahrq_ccs)))

# replace ahrq_ccs with procedure_group2
final_vars_categorical <- setdiff(final_vars_categorical, "ahrq_ccs")
final_vars_categorical <- c(final_vars_categorical, "procedure")
print(final_vars_categorical)
# check unique levels in each categorical variable
for (v in final_vars_categorical) {
  cat("Variable:", v, "\n")
  print(unique(data_clean[[v]]))
}
data_clean <- data_clean %>%
  mutate(
    procedure = factor(
      procedure,
      levels = c(
        "Other non-major procedures",  # reference
        "Gynecologic",
        "Urologic",
        "Orthopedic & Spine",
        "Gastrointestinal & Hernia"
      )
    ),
    dow = factor(
      dow,
      levels=c("1","2","3","4","5")
    ),
    month = factor(
      month,
      levels=c("1","2","3","4","5","6","7","8","9","10","11","12")
    )
  )
# drop original procedure_group and ahrq_ccs
```

```r
data_to_model <- data_clean %>%
  dplyr::select(-c(procedure_group, ahrq_ccs, "complication","moonphase","dow","month","ccsMort30Rate"))
str(data_to_model)

# -------------------------
# CREATE HOUR CATEGORIES
# -------------------------
data_to_model$hour_cat <- cut(
  data_to_model$hour,
  breaks = c(6, 8, 11, 13, 15, 17, 19),
  right = FALSE,  # intervals left-inclusive
  include.lowest = FALSE,
  labels = c("6-8am", "8-11am", "11-1pm", "1-3pm", "3-5pm", "5-7pm")
)

# check distribution
table(data_to_model$hour_cat, useNA = "ifany")
#return row for na hour_cat
data_to_model[is.na(data_to_model$hour_cat), ]
#if hour=19 set it to "5-7"
data_to_model$hour_cat[is.na(data_to_model$hour_cat) & data_to_model$hour == 19] <- "5-7pm"
table(data_to_model$hour_cat, useNA = "ifany")

##################################################
# Plot baseline characteristics table
##################################################
## ---------------------------------------------------
## 0. Pre-processing: factors, labels, 0/1 → No/Yes
## ---------------------------------------------------

# set your hour_cat levels if you haven't already
data_to_plot <- data_to_model %>%
  mutate(
    # gender: 0 = Male, 1 = Female
    gender = factor(ifelse(gender == 1, "Female", "Male"),
                    levels = c("Male", "Female")),
    mort30 = factor(ifelse(mort30 == 1, "Yes", "No"),
                    levels = c("No", "Yes")),
    asa_status = factor(asa_status,
                        levels = c("1", "2", "3"),
                        labels = c(
                          "I-II",
                          "III",
                          "IV-VI"
                        )
    )
  )

# list of binary baseline comorbidity variables that are 0/1
comorb_vars <- c(
  "baseline_cancer", "baseline_cvd", "baseline_dementia",
  "baseline_diabetes", "baseline_digestive",
  "baseline_osteoart", "baseline_psych", "baseline_pulmonary"
```

```r
)

# recode all of them 0/1 -> No/Yes
data_to_plot <- data_to_plot %>%
  mutate(
    across(all_of(comorb_vars),
           ~ factor(ifelse(. == 1, "Yes", "No"),
                    levels = c("No", "Yes")))
  )

## ---------------------------------------------------
## 1. Variable sets & ordering
## ---------------------------------------------------

cont_vars <- c("age", "mortality_rsi", "baseline_charlson")

cat_vars <- names(data_to_plot)[
  !(names(data_to_plot) %in% c(cont_vars,  "hour", "hour_cat"))
]

# put main variables first, then comorbidities, then any other cats
main_cat <- c("gender", "asa_status","procedure")
other_cat <- setdiff(cat_vars, c(main_cat, comorb_vars))

var_order <- c(
  "age",
  "mortality_rsi",
  "baseline_charlson",
  main_cat,
  comorb_vars,
  other_cat
)
var_order
## ---------------------------------------------------
## 2. gtsummary baseline table
## ---------------------------------------------------

tbl_baseline <-
  data_to_plot %>%
  dplyr::select(hour_cat, all_of(var_order)) %>%
  tbl_summary(
    by = hour_cat,
    statistic = list(
      all_continuous()  ~ "{mean} ± {sd}",
      all_categorical() ~ "{n} ({p}%)"
    ),
    missing = "no",
    label = list(
      age              ~ "Age",
      mortality_rsi    ~ "Risk Stratification Index (30-day mortality)",
      baseline_charlson~ "Charlson Comorbidity Index",
      gender           ~ "Gender",
      procedure        ~ "Procedure",
```

```r
      asa_status       ~ "ASA Physical Status",
      # comorbidities
      baseline_cancer      ~ "Cancer",
      baseline_cvd         ~ "Cardiovascular/Cerebrovascular disease",
      baseline_dementia    ~ "Dementia",
      baseline_diabetes    ~ "Diabetes",
      baseline_digestive   ~ "Digestive disease",
      baseline_osteoart    ~ "Osteoarthritis",
      baseline_psych       ~ "Psychiatric disorder",
      baseline_pulmonary   ~ "Pulmonary disease",
      mort30               ~ "30-day Mortality"
    )
  ) %>%
  add_overall(last = FALSE) %>%
  add_p(
    test = list(
      all_continuous()  ~ "oneway.test",
      all_categorical() ~ "chisq.test",
      baseline_dementia ~ "fisher.test",
      mort30            ~ "fisher.test"
    ),
    test.args = list(
      all_continuous()  ~ list(var.equal = TRUE),
      baseline_dementia ~ list(simulate.p.value = TRUE, B = 10000),
      mort30            ~ list(simulate.p.value = TRUE, B = 10000)
    )
  ) %>%
  modify_header(label ~ "**Variable**") %>%
  bold_labels()

## ---------------------------------------------------
## 3. Convert to gt and save PNG
##    (wide but not too tall)
## ---------------------------------------------------

gt_tbl <- as_gt(tbl_baseline) %>%
  gt::tab_style(
    style = gt::cell_fill(color = "#f7f7ec"),
    locations = gt::cells_body()
  ) %>%
  gt::tab_style(
    style = gt::cell_fill(color = "#f7f7ec"),
    locations = gt::cells_column_labels()
  ) %>%
  gt::opt_row_striping(
    row_striping = TRUE
  ) %>%
  gt::tab_options(
    table.font.size = 9,
    data_row.padding = gt::px(1),
    heading.padding = gt::px(2)
  )
gt_tbl
```

```r
##################################################
#Modeling code here
##################################################
##################################
#load data
##################################
data <- data_to_model
data$hour_cat <- relevel(data$hour_cat, ref = "8-11am")
str(data)
# check missingness for data
miss_summary <- naniar::miss_var_summary(data)
print(miss_summary)
names(data)
dim(data)

##################################
# build models
##################################
# set up
vars_spline <- c("age", "mortality_rsi")
var_exposure_cat <- "hour_cat"
dd <- datadist(data)
options(datadist = "dd")

# automatically define other covariates
other_covariates <- setdiff(
  names(data),
  c(var_outcome, var_exposure, vars_spline, var_exposure_cat)
)
print(other_covariates)
print(length(other_covariates))

# construct base rhs
base_rhs <- paste(
  "rcs(age, K) + rcs(mortality_rsi, K) + rcs(hour, K)",
  if (length(other_covariates) > 0)
    paste("+", paste(other_covariates, collapse = " + "))
  else ""
)

# fit models with different K
fit_k <- list()
AIC_k <- numeric()

for (K in c(0, 3, 4,5)) {
  if (K == 0) {
    # k = 0 = linear model: no rcs(), just linear terms
    f <- lrm(
      as.formula(
        paste("mort30 ~ age + mortality_rsi + hour",
              if (length(other_covariates) > 0)
                paste("+", paste(other_covariates, collapse = " + "))
              else "")
```

```r
      ),
      data = data
    )
  } else {
    f <- lrm(
      as.formula(paste("mort30 ~", gsub("K", K, base_rhs))),
      data = data
    )
  }
  fit_k[[as.character(K)]] <- f
  AIC_k[as.character(K)] <- round(AIC(f),0)
}

AIC_k
# choose the k with lowest AIC
best_k <- as.numeric(names(which.min(AIC_k)))
print(paste("Best K:", best_k))
best_model <- fit_k[[as.character(best_k)]]

# within best k, check which variables are truely nonlinear
#if nonlinear has small p keep spline, if p large change to linear
anova(best_model)

# -------------------------
# FIT MODEL WITH HOUR AS CATEGORICAL VARIABLE
# -------------------------
final_model_rhs <- as.formula(
  paste("mort30 ~ age + rcs(mortality_rsi,3) + hour_cat",
        if (length(other_covariates) > 0)
          paste("+", paste(other_covariates, collapse = " + "))
        else "")
)
fit_final_basic <- lrm(
  final_model_rhs,
  data = data
)
anova(fit_final_basic)

#check AIC for all models
c(
  AIC_k,
  final=round(AIC(fit_final_basic),0)
)

# unadjusted effect of hour
fit_unadjusted <- lrm(
  mort30 ~ hour_cat,
  data = data
)


#####################################################
# plots for report purposes
```

```r
#####################################################
# this object contains coeffients, se, z statistic, wald p-values, confidence intervals
## 1) Coefficients (betas)
co <- coef(fit_final_basic)

## 2) Variance-covariance matrix and SEs
vc <- vcov(fit_final_basic)
se <- sqrt(diag(vc))

## 3) Wald Z-statistics and p-values
z  <- co / se
p  <- 2 * pnorm(-abs(z))

## 4) Wald 95% CI for betas
ci_beta <- confint.default(fit_final_basic)  # works for lrm objects

# -------------- VARIABLE LABELS & LEVEL LABELS -------------------

## 5) Put everything together on OR scale
options(scipen = 999)   # prevents scientific notation
or_table <- data.frame(
  term    = names(co),
  beta    = round(co, 3),
  se      = round(se, 3),
  z       = round(z, 3),
  p_value = ifelse(p < 0.001, "<0.001", round(p, 3)),
  OR      = round(exp(co), 2),
  lower95 = round(exp(ci_beta[, 1]), 2),
  upper95 = round(exp(ci_beta[, 2]), 2),
  row.names = NULL,
  check.names = FALSE
)
or_table <- or_table %>%
  mutate(
    clean_term = dplyr::recode(term,

                               # continuous variables
                               "age"               = "Age",
                               "mortality_rsi"     = "Risk Stratification Index (30-day mortality)",
                               "baseline_charlson" = "Charlson Comorbidity Index",

                               # gender (binary)
                               "gender=1" = "Gender=Female",

                               # ASA levels
                               "asa_status=2" = "ASA status III",
                               "asa_status=3" = "ASA status IV-VI",

                               # Procedure categories
                               "procedure=GI"     = "Gastrointestinal & Hernia (vs other)",
                               "procedure=Ortho"  = "Orthopedic & Spine (vs other)",
                               "procedure=Gyn"    = "Gynecologic (vs other)",
                               "procedure=Uro"    = "Urologic (vs other)",
```

```r
                                # Binary comorbidities (0/1)
                                "baseline_cancer"    = "Cancer",
                                "baseline_cvd"       = "Cardiovascular/Cerebrovascular disease",
                                "baseline_dementia"  = "Dementia",
                                "baseline_diabetes"  = "Diabetes",
                                "baseline_digestive" = "Digestive disease",
                                "baseline_osteoart"  = "Osteoarthritis",
                                "baseline_psych"     = "Psychiatric disorder",
                                "baseline_pulmonary" = "Pulmonary disease",

                                # Hour categories
                                "hour_cat=8-11am"   = "Surgery Hour 8-11am",
                                "hour_cat=11-1pm" = "Surgery Hour 11-1pm",
                                "hour_cat=1-3pm"    = "Surgery Hour 1-3pm",
                                "hour_cat=3-5pm"    = "Surgery Hour 3-5pm",
                                "hour_cat=5-7pm"    = "Surgery Hour 5-7pm",
                                "hour_cat=6-8am"   = "Surgery Hour 6-8am",

                                .default = term    # fallback preserves anything unmapped
    )
  )


#significant variables at alpha = 0.05
or_table %>%
  filter(p_value != "<0.001" & p_value < 0.05 | p_value == "<0.001")

# 1. Start from your existing or_table
# or_table has: term, OR, lower95, upper95, p_value, etc.

plot_df <- or_table %>%
  # remove vector of terms don't want to plot
  filter(!term %in% c("Intercept","mortality_rsi","mortality_rsi'")) %>%
  filter(!is.na(OR)) %>%
  mutate(
    # numeric p-value
    p_num = 2 * pnorm(-abs(beta / se)),

    # significance flag
    signif = p_num < 0.05 &
      ((lower95 > 1) | (upper95 < 1)),

    signif_group = ifelse(
      signif,
      "Significant",
      "Not significant"
    ),

    # formatted p-value
    label_p = ifelse(
      p_num < 0.001,
      "p<0.001",
```

```r
      paste0("p=", sprintf("%.3f", p_num))
    ),

    # text label: ASA IV-VI gets 2 lines, others stay 1 line
    label = if_else(
      clean_term == "ASA status IV-VI",
      paste0(sprintf("%.2f", OR), "\n(", label_p, ")"),
      paste0(sprintf("%.2f", OR), " (", label_p, ")")
    )
  ) %>%
  arrange(OR) %>%
  mutate(clean_term = factor(clean_term, levels = clean_term))

plot_df <- plot_df %>%
  mutate(
    OR_text = sprintf("%.2f", OR),
    p_text  = paste0("(", label_p, ")"),
    v_OR = if_else(clean_term == "ASA status IV-VI",  0.3, 0),
    v_p  = if_else(clean_term == "ASA status IV-VI", -0.3, 0)
  )


ggplot(plot_df, aes(x = OR, y = clean_term, color = signif_group)) +
  geom_vline(xintercept = 1, linetype = "dashed", color = "grey40") +

  geom_errorbarh(aes(xmin = lower95, xmax = upper95), height = 0.15) +
  geom_point(size = 2) +

  geom_text(
    aes(x = upper95, label = label),
    hjust = -0.03,
    size = 3
  ) +

  scale_x_log10(expand = expansion(mult = c(0.05, 0.20))) +

  scale_color_manual(
    name = NULL,
    values = c(
      "Significant" = "#CC3311",
      "Not significant"    = "grey40"
    ),
    labels = c("Not significant", "Significant"),
    breaks = c(
      "Not significant",
      "Significant"
    )
  ) +

  guides(color = guide_legend(nrow = 1)) +

  labs(
    x = "Odds ratio (log scale)",
```

```r
    y = NULL
  ) +

  theme_light() +
  theme(
    panel.background  = element_rect(fill = "white", color = NA),
    panel.border      = element_blank(),
    panel.grid.major.x = element_blank(),
    panel.grid.minor.x = element_blank(),
    panel.grid.major.y = element_line(color = "grey85", linewidth = 0.4),
    panel.grid.minor.y = element_blank(),

    axis.line   = element_blank(),
    axis.ticks  = element_blank(),
    # x title no longer bold
    axis.title.x = element_text(size = 12, face = "plain"),
    axis.text   = element_text(size = 10, color = "black"),
    axis.text.y = element_text(size = 10, hjust = 0),

    legend.position = "top",
    legend.title    = element_blank(),
    legend.text     = element_text(size = 9),
    legend.key      = element_blank(),

    plot.background = element_rect(fill = "white", color = NA),
    plot.margin     = margin(2, 2, 2, 2)
  )
```

```r
# Get predictions for unadjusted and adjusted models
p1 <- as.data.frame(Predict(fit_unadjusted, hour_cat, fun = plogis)) %>%
  mutate(model = "Unadjusted")

p2 <- as.data.frame(Predict(fit_final_basic, hour_cat, fun = plogis)) %>%
  mutate(model = "Adjusted")

combined_plot <- bind_rows(p1, p2)
combined_plot$hour_cat_plot <- factor(
  combined_plot$hour_cat,
  levels = c("6-8am", "8-11am", "11-1pm", "1-3pm", "3-5pm", "5-7pm")
)
library(scales)  # for label_percent()

# Ensure hour_cat is a factor in the combined data too

ggplot(
  combined_plot,
  aes(x = hour_cat_plot, y = yhat, color = model, group = model)
) +
  # 95% CI ribbons
  geom_ribbon(
    aes(ymin = lower, ymax = upper, fill = model),
    alpha = 0.20,
    color = NA
```

```r
) +
# Lines for predicted probabilities
geom_line(size = 1.3) +

labs(
  x = "Surgery hour category",
  y = "Predicted 30-day mortality (%)"
) +

# Colors for unadjusted vs adjusted
scale_color_manual(
  name = NULL,
  values = c(
    "Unadjusted" = "#0BA6DF",
    "Adjusted"   = "#CC3311"
  )
) +
scale_fill_manual(
  name = NULL,
  values = c(
    "Unadjusted" = "#0BA6DF",
    "Adjusted"   = "#CC3311"
  )
) +

# Show y-axis as percentages (but keep underlying data as probabilities)
scale_y_continuous(
  labels = label_percent(accuracy = 0.01),  # e.g. 0.45%
  expand = expansion(mult = c(0, 0.05))
) +

# JAMA-ish theme
theme_bw() +
theme(
  panel.background   = element_rect(fill = "white", color = NA),
  panel.border       = element_blank(),
  panel.grid.major.x = element_blank(),
  panel.grid.minor.x = element_blank(),
  panel.grid.minor.y = element_blank(),
  panel.grid.major.y = element_line(color = "grey85", linewidth = 0.4),

  axis.line    = element_blank(),
  axis.ticks   = element_blank(),
  axis.title.x = element_text(size = 12),
  axis.title.y = element_text(size = 12),
  axis.text.x  = element_text(size = 10),
  axis.text.y  = element_text(size = 10),

  legend.position = "top",
  legend.title    = element_blank(),
  legend.key      = element_blank(),
  legend.text     = element_text(size = 10),
```

```r
    plot.background = element_rect(fill = "white", color = NA),
    plot.margin     = margin(5, 5, 5, 5)
  )
```

```r
#effect plot mortality_rsi
df_rsi <- as.data.frame(
  Predict(fit_final_basic, mortality_rsi, fun = plogis)
) %>%
  mutate(
    mort_pct = yhat * 100,
    lower_pct = lower * 100,
    upper_pct = upper * 100
  )

ggplot(df_rsi, aes(x = mortality_rsi, y = mort_pct)) +
  geom_line(size = 1.2, color = "#0BA6DF") +
  geom_ribbon(aes(ymin = lower_pct, ymax = upper_pct),
              alpha = 0.20, fill = "#0BA6DF") +
  labs(
    x = "Risk Stratification Index (30-day Mortality)",
    y = "Predicted 30-day Mortality (%)"
  ) +
  theme_bw() +
  theme(
    panel.grid.minor = element_blank(),
    axis.title = element_text(size = 12),
    axis.text = element_text(size = 10)
  )
```